ТЕТРИС НА C++

РАБОТУ ВЫПОЛНИЛИ: РОГОВ АНДРЕЙ,
ЯКОВЛЕВА НАСТЯ, САГУТДИНОВА ИЛОНА

Р Е А Л И З А Ц И Я

- Консольное приложение обеспечивает управление с поддержкой русскоязычной раскладки (клавиши WASD), динамическое начисление очков, автоматическую генерацию фигур и настройки скорости игры

СТРУКТУРА ИГРОВОГО ПОЛЯ И ПРИНЦИП ДВУХСВЯЗНЫХ СПИСКОВ

- Игровое поле состоит из 20 строк по 10 ячеек в каждой. Каждая строка представлена двусвязным списком ячеек для удобного доступа и управления содержимым.
- Все строки связаны между собой в двусвязный список, что обеспечивает эффективное добавление и удаление строк.

```
const int WIDTH = 10;
const int HEIGHT = 20;

struct Cell {
    char value = ' ';
    int color = WHITE;
    Cell* prev = nullptr;
    Cell* next = nullptr;
};

struct Row {
    Cell* head = nullptr;
    Cell* tail = nullptr;
    Row* prev = nullptr;
    Row* next = nullptr;
    int filled_count = 0;

    ~Row() {
        Cell* cur = head;
        while (cur) {
            Cell* nxt = cur->next;
            delete cur;
            cur = nxt;
        }
    }
};
```

ТИПЫ ФИГУР

- В игре используется семь фигур с соответствующими цветами в массиве TETRAMINO_SHAPES, цвета заданы константами ENUM Colors.
- В начале игры создаются две фигуры – текущая и следующая, которая генерируется случайно с выбором типа и цвета.

```
enum Colors {
    BLACK = 0, DARKYELLOW = 6, WHITE = 7, BLUE = 9, GREEN = 10, CYAN = 11, RED = 12, MAGENTA = 13, YELLOW = 14
};
const Colors TETRAMINO_COLORS[7] = { CYAN, YELLOW, MAGENTA, GREEN, RED, BLUE, DARKYELLOW };

// Массив фигур (7 типов фигур, каждая с 4 поворотами, каждая фигура состоит из 4 блоков с координатами [x, y] относительно центра)
int TETRAMINO_SHAPES[7][4][4][2] = {
    // I
    {{{-1,0},{0,0},{1,0},{2,0}}, {{0,-1},{0,0},{0,1},{0,2}}, {{-1,0},{0,0},{1,0},{2,0}}, {{0,-1},{0,0},{0,1},{0,2}}},
    // O
    {{{-1,-1},{0,-1},{-1,0},{0,0}}, {{-1,-1},{0,-1},{-1,0},{0,0}}, {{-1,-1},{0,-1},{-1,0},{0,0}}, {{-1,-1},{0,-1},{-1,0},{0,0}}},
    // T
    {{{-1,0},{0,0},{1,0},{0,-1}}, {{0,-1},{0,0},{0,1},{1,0}}, {{-1,0},{0,0},{1,0},{0,1}}, {{0,-1},{0,0},{0,1},{-1,0}}},
    // S
    {{{-1,0},{0,0},{0,-1},{1,-1}}, {{0,-1},{0,0},{1,0},{1,1}}, {{-1,0},{0,0},{0,-1},{1,-1}}, {{0,-1},{0,0},{1,0},{1,1}}},
    // Z
    {{{-1,-1},{0,-1},{0,0},{1,0}}, {{0,-1},{0,0},{-1,0},{-1,1}}, {{-1,-1},{0,-1},{0,0},{1,0}}, {{0,-1},{0,0},{-1,0},{-1,1}}},
    // J
    {{{-1,0},{0,0},{1,0},{-1,-1}}, {{0,-1},{0,0},{0,1},{-1,1}}, {{-1,0},{0,0},{1,0},{1,1}}, {{0,-1},{0,0},{0,1},{1,-1}}},
    // L
    {{{-1,0},{0,0},{1,0},{1,-1}}, {{0,-1},{0,0},{0,1},{1,1}}, {{-1,0},{0,0},{1,0},{-1,1}}, {{0,-1},{0,0},{0,1},{-1,-1}}}
};
```

ПРОВЕРКА ПОЗИЦИИ И ПЕРЕМЕЩЕНИЯ ФИГУР

- Функция isValidPosition проверяет, можно ли переместить фигуру с учётом границ поля и занятости ячеек другими фигурами.
- Движение влево и вправо осуществляется через moveLeft и moveRight, проверяющие допустимость сдвига и обновляющие позицию фигуры.
- Метод dropSoft позволяет фигуре опускаться на одну строку вниз плавно, а dropHard быстро опускает её.

ВРАЩЕНИЕ И ПРОЕКЦИЯ (ТЕНЬ) ФИГУРЫ

- Метод rotate изменяет поворот фигуры по часовой стрелке.
- Функция getShadowY вычисляет максимальную вертикальную позицию, на которой фигура может оказаться при падении, для отображения её тени на поле.